TECHNICAL REPORT TR-2026-01

Silent-Failure Study · Week 1

A Library-Independent Reference Oracle for Cross-Engine Dynamics Adjudication

Noah Parsons

22 August 2026

Engine under measurement: MechanicsDSL 2.1.3, commit `a8dc2b2` (tag `v2.1.3`), frozen 22 August 2026.

Artefacts: `stress_suite/reference.py`, `stress_suite/validate_reference.py`. Committed at `584b3eb`.

Disclosure: the author maintains one of the engines measured. AI assistance was used in this work. Both are disclosed in Section 9.

Abstract

The silent-failure study compares three dynamics engines on the same mechanical systems, and adjudicates disagreement with an independent derivation of the equations of motion. The existing oracle is built on `sympy.physics.mechanics`, which is adequate while MechanicsDSL is the only engine under test but becomes near-circular once SymPy itself is a contestant. This report describes a replacement oracle derived in closed form using `numpy` alone, sharing no library with any engine under test. The reference reproduces the equations of motion of all three portable test families, is validated by eleven internal consistency checks, and agrees with both the existing SymPy oracle and with MechanicsDSL to machine precision ($\leq 5.5 \times 10^{-15}$ relative) across every case both engines can compute. Independent adjudication coverage rises from 22 to 31 of the suite's 55 cases, and for the first time extends to the Hamiltonian pathway. The report also records a false positive encountered during validation — an apparent silent failure of magnitude 4.53 that proved to be a state-convention mismatch — together with the diagnostic that distinguished it, since the same trap is latent in the existing harness. Three incidental findings about the study's scope and about the engine's scaling behaviour are documented in Section 7.

Contents

1 Motivation

A verdict in this study rests on comparing an engine's answer against an independent derivation. The independence of that derivation is therefore the load-bearing assumption, and it does not survive the study's own design.

The current oracle, `groundtruth.py`, derives the equations of motion with `sympy.physics.mechanics.Lagrange` Measured against MechanicsDSL this is a genuinely separate code path and a fair referee. But the study's second engine *is* `sympy.physics.mechanics`. Adjudicating that column with a hand-rolled SymPy derivation would mean checking SymPy against SymPy: different functions, but shared `diff`, shared `simplify`, shared `solve`, and a shared assumptions system underneath. Agreement under those conditions is closer to a self-consistency check than to confirmation.

The consequence is not merely a weak check on one column. It is *asymmetric evidence across columns* — one adjudicated independently, another adjudicated by something weaker, reported side by side under a single heading. A reviewer need not demonstrate an error to reject such a table; it is enough to observe the asymmetry.

A second, independent motivation is coverage. The SymPy oracle applies only to the unconstrained Lagrangian pathway. Of the suite's 55 cases, 33 (60 %) carry no independent derivation check at all, and the gap coincides with the timeout wall: the Hamiltonian and constrained pathways are both thinly covered and go blind exactly where the systems become difficult.

Both problems have one solution: a reference that shares no library with any contestant, and that works from state rather than from symbolic equations.

2 The reference

2.1 Scope

The suite's portable axes rest on three distinct physical systems (Section 7.1). All three are implemented:

Axis	System	Class
`dof`	Planar N-link pendulum chain	`NLinkChain`
`near_singular`	Two masses, near-degenerate coupling	`LinearSystem`
`mass_ratio`	Two masses and springs, ratio 10^k	`LinearSystem`

2.2 Derivation for the chain

The chain has N point masses m on massless rods of length l, angles measured from the downward vertical. Its mass matrix follows directly from the kinetic term,

$$M_{ij}(\theta) = a_{ij}\cos(\theta_i - \theta_j), \qquad a_{ij} = \big(N - \max(i,j)\big)\, m\, l^2. \tag{1}$$

For a Lagrangian $L = \frac{1}{2}\dot\theta^\top M(\theta)\dot\theta - V(\theta)$, the Euler–Lagrange equations give

$$\sum_j M_{ij}\ddot\theta_j + \sum_{j,k}\Big[\frac{\partial M_{ij}}{\partial\theta_k} - \frac{1}{2}\frac{\partial M_{jk}}{\partial\theta_i}\Big]\dot\theta_j\dot\theta_k + \frac{\partial V}{\partial\theta_i} = 0. \tag{2}$$

Substituting $\partial M_{ij}/\partial\theta_k = -a_{ij}\sin(\theta_i - \theta_j)(\delta_{ik} - \delta_{jk})$ into the bracket, the cross terms cancel and the double sum collapses to a single sum of squared velocities:

$$C_i(\theta,\dot\theta) = \sum_j a_{ij}\sin(\theta_i - \theta_j)\,\dot\theta_j^2. \tag{3}$$

With $G_i(\theta) = (N-i)\, mgl\sin\theta_i$, the equation of motion is $M\ddot\theta + C + G = 0$, solved numerically for $\ddot\theta$ at each evaluation. The mass matrix is never inverted symbolically: that is precisely the operation which fails to scale in the engines under test, and performing it in the referee would import their scaling limit into the adjudicator.

The cancellation yielding (3) is the reason this family is worth deriving by hand at all — it reduces to a few lines of array arithmetic. It is also the step at which a sign error would hide, and is therefore verified numerically rather than trusted (Section 3).

2.3 The canonical pathway

The engine's Hamiltonian pathway integrates (q, p) and returns $(\dot{q}, \dot{p})$. To be comparable there, the reference also provides

$$p = M(\theta)\dot{\theta}, \qquad \dot{p} = M\ddot{\theta} + \dot{M}\dot{\theta}, \qquad \dot{M}_{ij} = -a_{ij}\sin(\theta_i - \theta_j)(\dot{\theta}_i - \dot{\theta}_j), \tag{4}$$

together with $H = \frac{1}{2}p^\top M^{-1} p + V$. Deriving $\dot{p}$ from $p = M\dot{\theta}$ rather than from H directly keeps the two sides mutually consistent by construction; they are cross-checked against each other in Section 3.

2.4 The linear systems

Both remaining portable systems reduce to $M\ddot{q} + Kq = 0$ with constant coefficients:

$$\texttt{near\_singular}(\varepsilon): \quad M = m\begin{pmatrix} 1 & 1-\varepsilon \\ 1-\varepsilon & 1 \end{pmatrix}, \qquad K = k\,I, \tag{5}$$

$$\texttt{mass\_ratio}(r): \quad M = \mathrm{diag}(m_1,\, r), \qquad K = \begin{pmatrix} k + k_1 & -k \\ -k & k \end{pmatrix}. \tag{6}$$

Here $\det M = m^2(2\varepsilon - \varepsilon^2)$, vanishing exactly at $\varepsilon = 0$ — the genuinely degenerate case the engine is expected to refuse. A singular mass matrix raises rather than falling back to a least-squares solution, since substituting a well-defined answer to a different question is the exact failure mode this study measures.

3 Internal validation

Eleven checks run via `python reference.py`; all pass.

Check	Result
$N = 1$ against the analytic simple pendulum	exact to 10^{-13}
$N = 2$ against a separately hand-written double pendulum	worst 3.55×10^{-15} over 500 random states
Coriolis identity (3) against finite-difference evaluation of the bracket in (2)	agrees for $N = 2, 3, 5$
Mass matrix symmetry and positive definiteness	holds
Small-oscillation frequencies against the generalised eigenproblem	agrees for $N = 1 \ldots 4$
Energy conservation under integration	5.1×10^{-12}, 2.5×10^{-13}, 3.0×10^{-13} for $N = 1, 2, 3$
$(q, \dot{q}) \leftrightarrow (q, p)$ roundtrip; $H = E$	exact
$\dot{p}$ against $-\partial H/\partial q$ by finite differences	agrees for $N = 1 \ldots 4$
Near-singular determinant law; refusal at $\varepsilon = 0$	holds; raises
Mass-ratio modes and energy at $r = 10^0, 10^6, 10^{12}$	positive definite; drift $< 10^{-8}$
Dispatch over the suite's case list	19 of 36 systems

The double-pendulum comparison is written from the standard textbook form rather than from the module's general code, so agreement is a genuine cross-check rather than a restatement.

One check initially failed at $\varepsilon = 10^{-8}$ because $\det M = 1-(1-\varepsilon)^2$ suffers catastrophic cancellation: the naive floating-point evaluation carries absolute error of order 10^{-16} irrespective of ε, so the relative error is already $\sim 5\times 10^{-9}$ there. The tolerance was wrong, not the reference. The loss is a property of the system — indeed it is the conditioning that axis exists to probe — and the test now bounds absolute rather than relative error.

4 Cross-validation

Three derivations were compared at 24 pseudorandom states per case (seed `20260822`), states drawn uniformly from $[-0.5, 0.5]$: the closed-form reference, the SymPy oracle, and MechanicsDSL at the frozen pin. Each pathway is compared on its own terms — full right-hand side against full right-hand side, in the coordinates that pathway uses.

Pathway	N	**ref ~ engine**	**ref ~ sympy**	**sympy ~ engine**
lagrangian	1	0.000	0.000	0.000
lagrangian	2	1.78×10^{-15}	0.000	1.78×10^{-15}
lagrangian	3	1.89×10^{-15}	1.78×10^{-15}	1.60×10^{-15}
lagrangian	4	0.000	0.000	0.000
lagrangian	5	5.54×10^{-15}	5.54×10^{-15}	9.42×10^{-16}
hamiltonian	1	0.000	—	—
hamiltonian	2	7.77×10^{-16}	—	—
hamiltonian	≥ 3	engine does not return; see Section 7.2		

All three derivations agree to machine precision wherever all three can be evaluated. Because the reference and the SymPy oracle share no library, their agreement validates the closed-form algebra independently of SymPy; and MechanicsDSL agreeing with both is the strongest statement the suite can presently make about the engine on these families.

5 A false positive, and the diagnostic that caught it

The first Hamiltonian comparison reported a relative mismatch of **4.53** on `dof_N2` — a case the baseline scores `pass`, on the pathway with no independent coverage, while the two reference derivations agreed with each other exactly. It presented as a textbook silent failure. It was not one.

The engine's Hamiltonian pathway integrates (q, p) and returns $(\dot{q}, \dot{p})$. Extracting the odd entries of the right-hand side and treating them as accelerations therefore compares $\dot{p}$ against $\ddot{q}$.

The diagnostic was the failure's dependence on system size: $N = 1$ agreed exactly while $N = 2$ did not. A genuine derivation error rarely switches on at a specific N; a convention mismatch does, because degenerate cases make different conventions coincide. Here $M = ml^2 = 1$ at $N = 1$, making p and $\dot{\theta}$ numerically identical. The confirming test distinguishes the conventions directly: under the Lagrangian convention $\dot{q}$ must equal the state's own odd entries, whereas under the canonical convention it equals $M^{-1}p$. Observed at $N = 2$:

```
dy[0::2] = [0.028159 0.422372]
state_odd = [0.450464 0.448649] <- Lagrangian convention: no
Minv*odd = [0.028159 0.422372] <- canonical convention: yes, exactly
```

With the reference extended to momenta, the mismatch fell from 4.53 to 7.77×10^{-16}.

The same assumption is latent in the harness. `worker.py` extracts `dydt[1::2]` and documents it as "the accelerations." This is correct on the Lagrangian pathway and a category error on the canonical one. It produces no incorrect verdicts at present only because the oracle never runs on that pathway — but extending coverage there in the obvious way, which is the study's stated next improvement, would yield mismatches of order unity and a reported silent failure that does not exist. In a study whose subject is false negatives, publishing a false positive of this kind would be difficult to recover from.

6 Coverage

Cases with independent adjudication	Before	After
`dof` (both pathways)	5	10
`near_singular`	7	7
`mass_ratio` (both pathways)	7	14
Other axes	3	0*
Total, of 55	**22**	**31**

*The reference deliberately does not cover `symbolic`, `redundancy`, or `loops`; the first two are not portable across engines, and the third is an open scope question.

The gain is not only in count. The prior oracle could adjudicate the Lagrangian pathway alone; the reference adjudicates any pathway that can be asked for a state derivative, which closes the Hamiltonian gap identified as the suite's highest-value outstanding improvement.

7 Incidental findings

7.1 The study's scope document describes a different design

`SCOPE.md` states that the study uses *one* test family — a pendulum chain — with four knobs dialled on it. The implemented suite does not. Only the `dof` axis is that chain; `near_singular` and `mass_ratio` are two-degree-of-freedom Cartesian systems with springs. These are different physical systems, not one system with different parameters.

This is not a defect in the suite. All three remain portable across engines, which is what the cross-engine claim requires, and both spring systems are linear with constant mass matrices, so exact references for them are trivial — which is why coverage rose to 31 cases rather than 10. But the scope document makes a false statement about the study it governs, and that statement would otherwise propagate into the paper.

7.2 The Hamiltonian scaling wall is at $N = 3$

The baseline records `dof_N3`, `dof_N4`, and `dof_N5` on the Hamiltonian pathway as timeouts at the 180 s limit. Re-running `dof_N3` for this report with a 400 s budget did not change the outcome. The wall is therefore a property of the engine on that pathway rather than an artefact of the chosen limit, at least up to 2.2× the budget — a useful answer to the predictable reviewer question of whether a longer timeout would alter the reported numbers.

7.3 Cost on the Lagrangian pathway is non-monotonic in N

	$N = 3$	$N = 4$	$N = 5$
Wall-clock (Lagrangian)	56.0 s	17.9 s	20.9 s

The three-link system costs roughly three times the five-link system. This is the symbolic-to-numeric threshold documented as fix 3 in `FIXES.md`: past a size bound the engine stops inverting the mass matrix symbolically and solves $M\ddot{q} = F$ numerically instead. $N = 3$ is the last case on the symbolic side. The behaviour contradicts the natural assumption that cost rises with system size and should be stated explicitly in any scaling discussion.

8 Status and open decisions

Complete. The reference is implemented, internally validated, cross-validated against two independent derivations, and committed at `584b3eb`. It is additive to the harness: it adds adjudication where there was none and alters no existing verdict, satisfying the re-validation condition on harness growth.

Outstanding, and required before the SymPy adapter:

1. **Adapter fidelity.** The SymPy column may be written *idiomatically* — using `LagrangesMethod` as the documentation directs, and scoring the result — or *carefully*, avoiding symbolic inversion as `groundtruth.py` does. The former measures the tool as its users encounter it, which matches the study's question, but may record failures a careful user would avoid; that must then be disclosed. Undecided.
2. **The `loops` axis.** Whether closed kinematic chains enter the cross-engine claim. Drake supports them and SymPy can express them via Lagrange multipliers, so the axis is arguably portable. Admitting it adds three constrained cases to a pathway with no independent coverage. Undecided.
3. **Integrator pinning.** Drake's default integrator and error control differ from `scipy`'s. On pathways where energy drift is the only oracle, comparing drift across engines risks measuring the integrator rather than the engine. Either pin one integrator family and tolerance across all three, or confine cross-engine claims to pathways the reference adjudicates. Undecided.

9 Disclosure

The author is the maintainer of MechanicsDSL, one of the three engines under measurement. This is disclosed here and in every document of the study. The mitigation is structural: verdicts rest on independent derivations and on cross-engine disagreement, never on an engine's self-report, and the work of this report strengthens that mitigation by removing the study's remaining dependence on a shared symbolic library.

AI assistance was used in the derivation, implementation, validation, and preparation of this report. All numerical results reported here were produced by executing the committed code and are reproducible via the commands in Section 10.

10 Reproduction

```
git checkout v2.1.3
cd stress_suite
python reference.py # eleven internal self-tests
python validate_reference.py # three-way cross-validation
```

Both scripts prepend the repository's `src/` directory to the module search path and abort if the engine resolves elsewhere, guarding against a stale installed copy being validated in place of the pinned source.